

# Fabled — Claude Code Context

You are building **Fabled**, a whimsical book club web app. Read this file fully before every session. Reference `/docs/PRD.md` for product logic, `/docs/DESIGN_SYSTEM.md` for all visual rules, and `/docs/archetypes.json` for archetype data. Never paraphrase or reinterpret archetype copy — use the JSON directly.

The person you are working with is non-technical. Adjust every response accordingly.

---

## Before You Write Any Code

Always do this first, every session:

1. State in plain English what you are about to build
2. List the exact files you will create or modify
3. Flag any risk or decision the user should know about before you proceed
4. Wait for the user to confirm before writing a single line of code

Example of the right format:

```
"I'm going to build the aura generation screen. I'll create 3 files: the API route that calls Anthropic, the loading screen component, and the aura card component. One thing to confirm: do you want the aura card to appear on the same page as the loading screen, or navigate to a new page?"
```

---

## After Every Task

When you finish a task, always provide:

1. **What was built:** 2-3 sentences in plain English, no jargon
2. **How to test it:** Numbered steps a non-technical person can follow on their phone. Be specific. "Click the button" is not enough — say which button, what screen, what should happen.
3. **What's not done yet:** Anything intentionally left incomplete or that needs follow-up

4. **Deploy check:** Confirm whether a Vercel deploy preview is needed before the next session

Example of the right format:

"I built the sign-up screen. It collects your email and display name and creates an account in Supabase.

To test: Open the app on your phone. Tap 'Get Started'. Enter your email and a display name. Tap 'Create Account'. You should see a confirmation message and land on the book input screen. If you see a red error message instead, screenshot it and share it with me.

Not done yet: Magic link sign-in is not wired up, only password auth works for now.

Deploy: Yes, deploy to Vercel before the next session so you can test on your actual phone."

---

## Stack

- **Framework:** Next.js 14+ with App Router
  - **Styling:** Tailwind CSS only. No external UI component libraries (no shadcn, no MUI, no Radix).
  - **Database + Auth:** Supabase (email/password + magic link)
  - **Realtime:** Supabase Realtime for tribe state — never use polling
  - **AI:** Anthropic API via server-side API routes ONLY. Model: `claude-sonnet-4-20250514`
  - **Hosting:** Vercel
  - **Fonts:** Google Fonts — Playfair Display, DM Sans, JetBrains Mono
- 

## Hard Rules — Never Break These

- **NEVER expose the Anthropic API key client-side.** All AI calls go through `/api/*` server routes. If you are ever unsure whether a key is exposed, check first.
- **NEVER use `<form>` tags.** Use `onClick` / `onChange` handlers.
- **NEVER use `localStorage` or `sessionStorage`.** All state lives in Supabase or React state.

- **NEVER build more than one feature per session.** Complete one thing fully before starting the next. The user needs to be able to test each piece independently.
  - **NEVER skip error states.** Every screen that makes an AI call or database call needs a visible error state and a retry mechanism.
  - **NEVER leave a session with broken code.** If something is half-built, say so explicitly and either finish it or roll it back.
- 

## Security Rules (Non-Negotiable)

These protect the user from unexpected costs and data exposure:

- **Rate limit all AI routes.** Add a simple rate limit to `/api/generate-aura`, `/api/tribes/[id]/generate-deck`, and `/api/tribes/[id]/discussion`. Maximum 10 requests per IP per hour. Use Vercel's built-in rate limiting or a simple in-memory approach.
  - **Supabase Row Level Security (RLS) must be enabled** on every table. Users should only be able to read and write their own data. Tribe data is readable by tribe members only.
  - **Validate all inputs server-side** before passing to AI or database. Never trust client-sent data.
  - **Never expose raw error messages to the client.** Log errors server-side. Show the user a generic friendly message.
- 

## Mobile-First Rules

- Design and build at 390px viewport width first
  - Minimum touch target: 44px x 44px on all interactive elements
  - Safe area bottom padding: 32px on all fixed bottom elements (home indicator)
  - Test swipe interactions with pointer events (works for touch and mouse)
  - Never use hover-only states — every interaction must work on tap
- 

## Architecture Rules

- App Router with server components by default. Only add `"use client"` when needed for interactivity or browser APIs.
  - Supabase: server client in API routes and server components, browser client only in client components.
  - Tribe state machine is the single source of truth. All UI derives from the tribe status enum: `empty | ready | swiping | reveal | voting | reading`. Never derive state from other signals.
  - Archetype data lives in `/docs/archetypes.json`. Import it directly. Never hardcode archetype strings or copy in components.
  - SVG creature illustrations are unique per archetype, rendered in the user's three aura colors. Build as React SVG components with color props.
- 

## AI Call Rules

- All AI prompts are in `/docs/AI_PROMPTS.md`. Use them verbatim. Do not modify.
  - Every AI call returns JSON only. Always wrap in try/catch. Always validate shape.
  - Strip accidental markdown fencing before parsing: `.replace(/```json|```/g, "").trim()`
  - If an AI call fails, show the user a friendly retry screen. Never lose their input data.
- 

## Scope Rules — Read Carefully

**Do not touch files outside the scope of the current task.** If you think a refactor elsewhere would help, flag it and ask first. Do not do it silently.

**Do not add features not in the PRD** without explicitly flagging it as an addition and getting confirmation.

**If you are uncertain about something**, stop and ask. A clarifying question is always better than a wrong assumption that takes three sessions to undo.

---

## Build Order

Work through these phases in order. Complete and deploy each before starting the next.

1. **Scaffold:** Next.js setup, Tailwind config, Supabase connection, font imports
  2. **Database:** Run all schema migrations, enable RLS on all tables
  3. **Auth:** Sign up, sign in, magic link, session handling, protected routes
  4. **Aura generation:** Book input → AI call → aura card display → save to Supabase
  5. **Home screen:** Aura mini-card, empty tribes state, bottom nav
  6. **Tribes:** Create tribe, join tribe, invite code/link, tribe card on home screen
  7. **Swipe deck:** Deck generation, swipe UI, vote recording, waiting room
  8. **Reveal + voting:** Match tiers, reveal animation, book voting, state transition
  9. **Discussion mode:** Question generation, four-tier UI, hot take refresh
  10. **Aura profiles:** Full aura screen, member modal, axes bars, compatibility
- 

## What Fabled Is

A mobile-first web app where readers:

1. Generate a "reading aura" personality (one of 12 archetypes) based on books they love
2. Form tribes with friends (groups of 2-12)
3. Swipe on AI-curated blind book descriptions
4. Vote on what to read next
5. Discuss with AI-generated conversation prompts

Tone: tarot card meets fantasy library meets book club. Mystical but not precious.

Whimsical but not childish. Dark, warm, intimate.

---

## Key Files

- `/docs/PRD.md` — Full product requirements and user stories
- `/docs/DESIGN_SYSTEM.md` — All design tokens, components, animations (use exact values)
- `/docs/archetypes.json` — All 12 archetypes with axes, compatibility, copy, visual direction

- `/docs/AI_PROMPTS.md` — All AI prompt templates (use verbatim)